Notes:

- HW6 due Nov 27 (All github classroom)
- HW7 due Dec 4 (All gradescope)
- Final project reports due Dec 4
- Email me before Tuesday's lecture if you want to volunteer to present on Dec. 2. Otherwise, I will randomly assign students to each day's presentation slots (see Brightspace announcement)

Last class:

- numpy datatype, `ndarray`
  - creating arrays
  - native operations
  - matrix multiplication
  - slicing
  - views and copies
- `scipy.constants`
- `scipy.special`

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
```

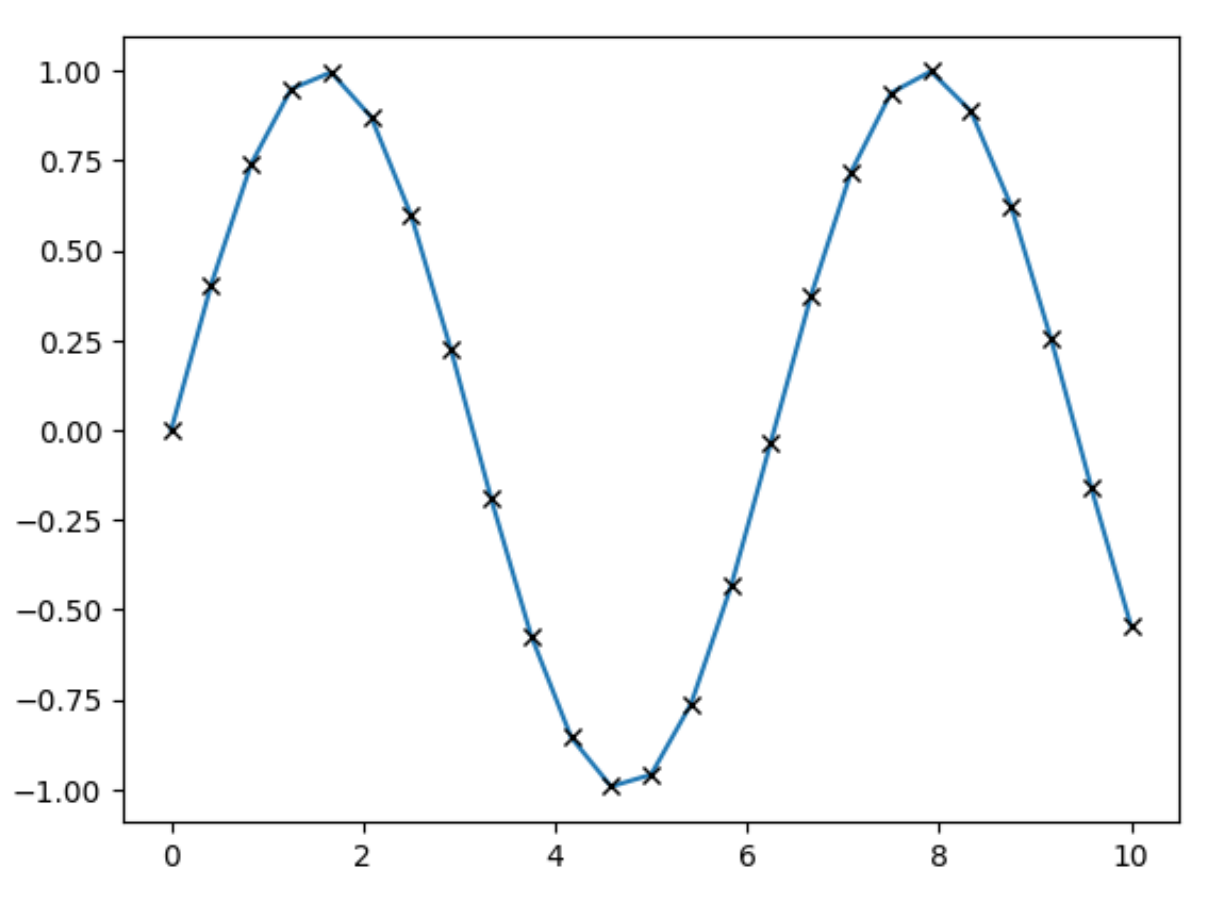
`scipy.interpolate` provides functions for interpolating data in 1 and many dimensions.

Most routines in `scipy.interpolate` take interpolation data as their input (such as known data points in (x_i, y_i) pairs) and produce *functions* as their output, that can be evaluated to produce interpolated data.

Simplest routine is `interp1d`, which takes two ndarrays as inputs, giving lists of x and y values and producing such a function:

In [2]:
```
import scipy.interpolate as interp

x = np.linspace(0,10,25)
y = np.sin(x)

f = interp.interp1d(x,y)

x_fine = np.linspace(0,10,500)
y_fine = f(x_fine)

plt.plot(x_fine, y_fine)
plt.plot(x, y, 'kx ')
```

Out[2]:
```
[<matplotlib.lines.Line2D at 0x12323d090>]
```

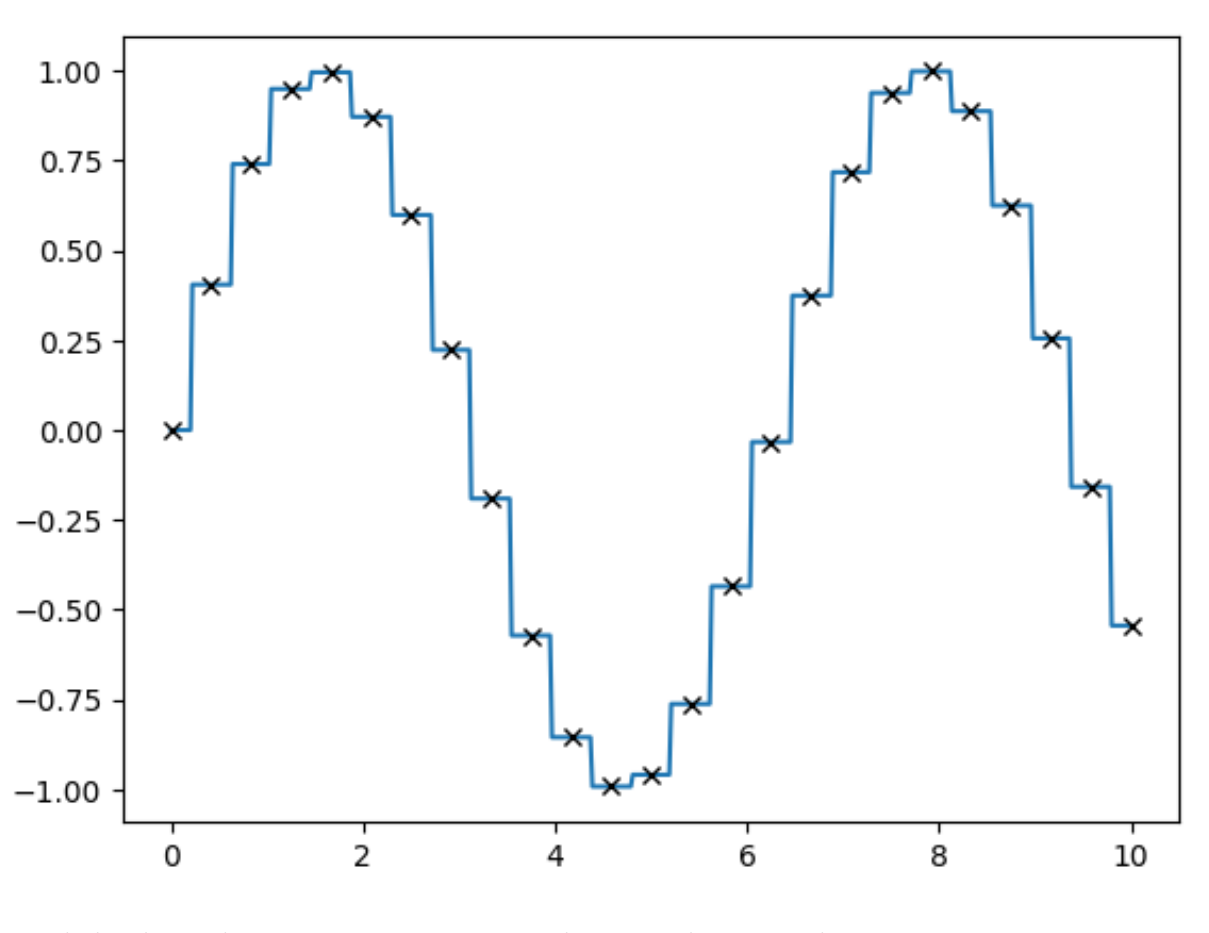


Gives piecewise linear interpolation of given data.

Can make this lower-order:

In [3]:
```
f = interp.interp1d(x, y, kind='nearest')
y_fine = f(x_fine)

plt.plot(x_fine, y_fine)
plt.plot(x, y, 'kx ')
```

Out[3]:
```
[<matplotlib.lines.Line2D at 0x1233ea990>]
```

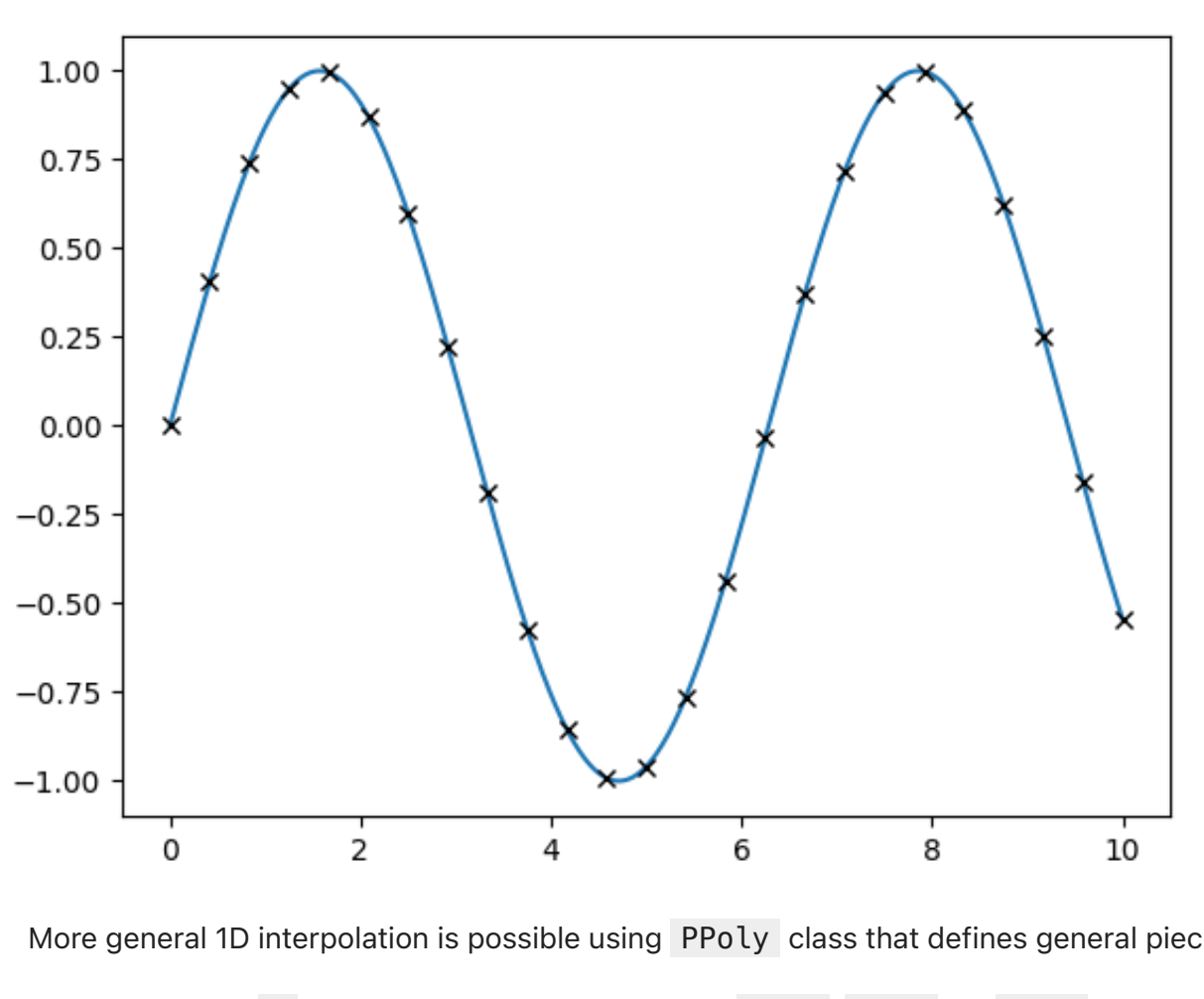


This is piecewise-constant (nearest-neighbour) interpolation.

Can also go higher order

In [4]:
```
f = interp.interp1d(x, y, kind='cubic')
y_fine = f(x_fine)

plt.plot(x_fine, y_fine)
plt.plot(x, y, 'kx ')
```

Out[4]:
```
[<matplotlib.lines.Line2D at 0x123462ad0>]
```



More general 1D interpolation is possible using `PPoly` class that defines general piecewise polynomial interpolation.

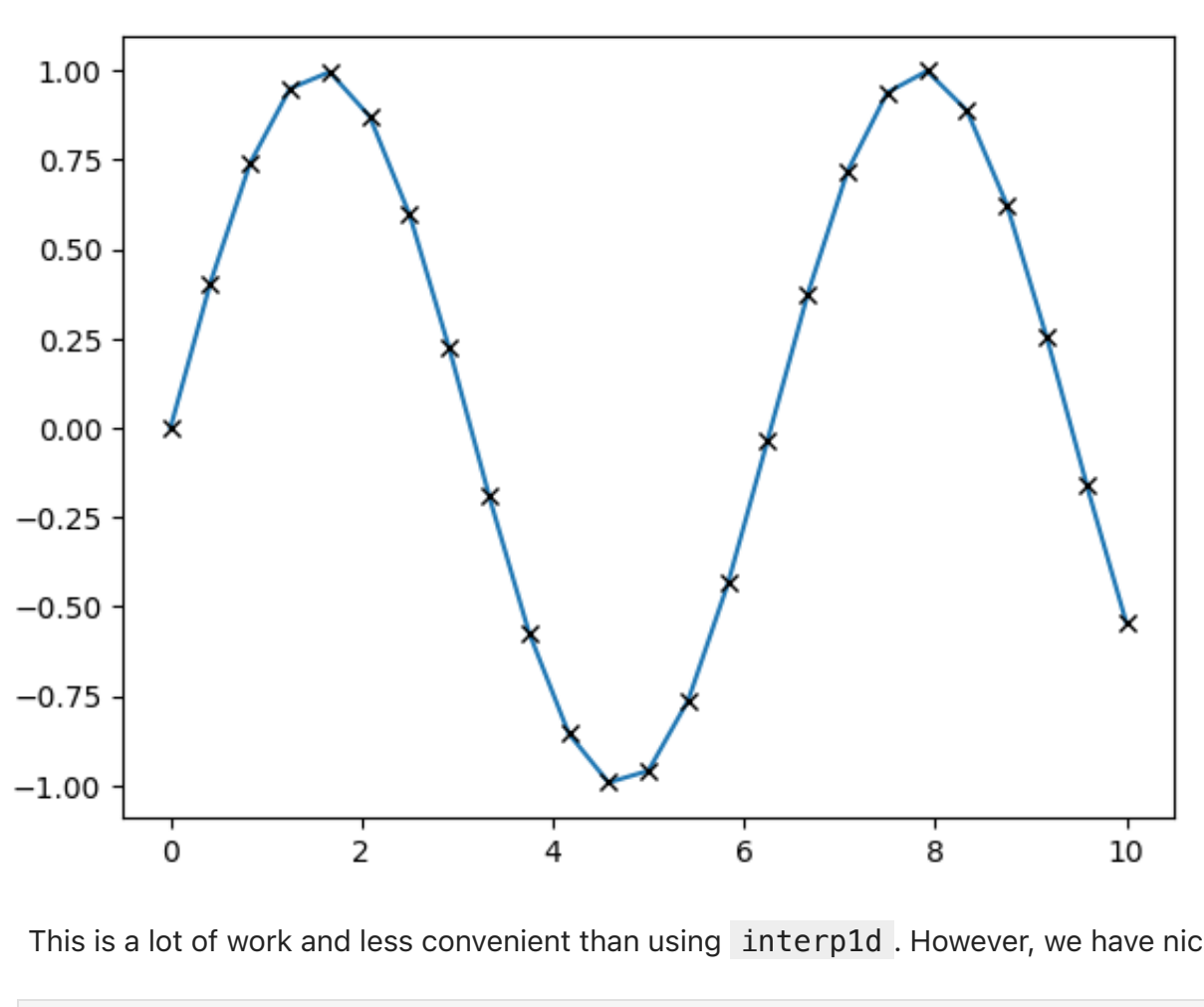
For this, we take `x` to be an array of $m+1$ points, `x[0]`, `x[1]`, ..., `x[m]`, that define m subintervals

Specify $k+1$ coefficients on each subinterval to get polynomial of degree k on each subinterval. Need $(k+1)m$ values in total

To get piecewise linear interpolation of (x, y) values, have to reconstruct linear representation on each interval

In [5]:
```
m = np.size(x)-1
c = np.zeros((2,m))

c[0,:] = (y[1:] - y[:-1])/(x[1:] - x[:-1])
c[1,:] = y[:-1]
g = interp.PPoly(c,x)

y_fine = g(x_fine)

plt.plot(x_fine, y_fine)
plt.plot(x, y, 'kx ')
```

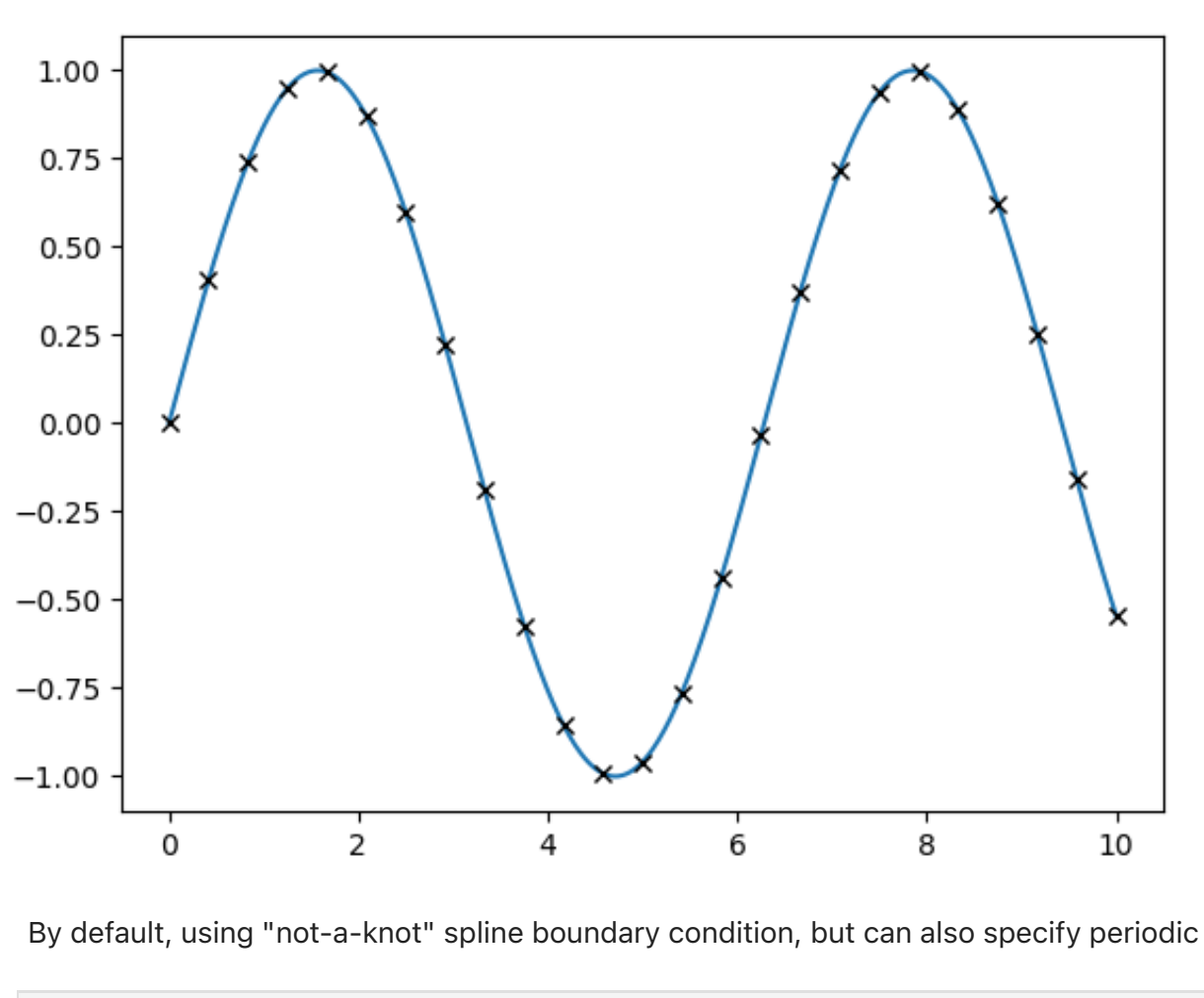
Out[5]:
```
[<matplotlib.lines.Line2D at 0x1235260d0>]
```



This is a lot of work and less convenient than using `interp1d`. However, we have nicer higher-order interpolation functions that build on top of `PPoly`

In [6]:
```
gspline = interp.CubicSpline(x,y)
y_fine = gspline(x_fine)
plt.plot(x_fine, y_fine)
plt.plot(x, y, 'kx ')
```

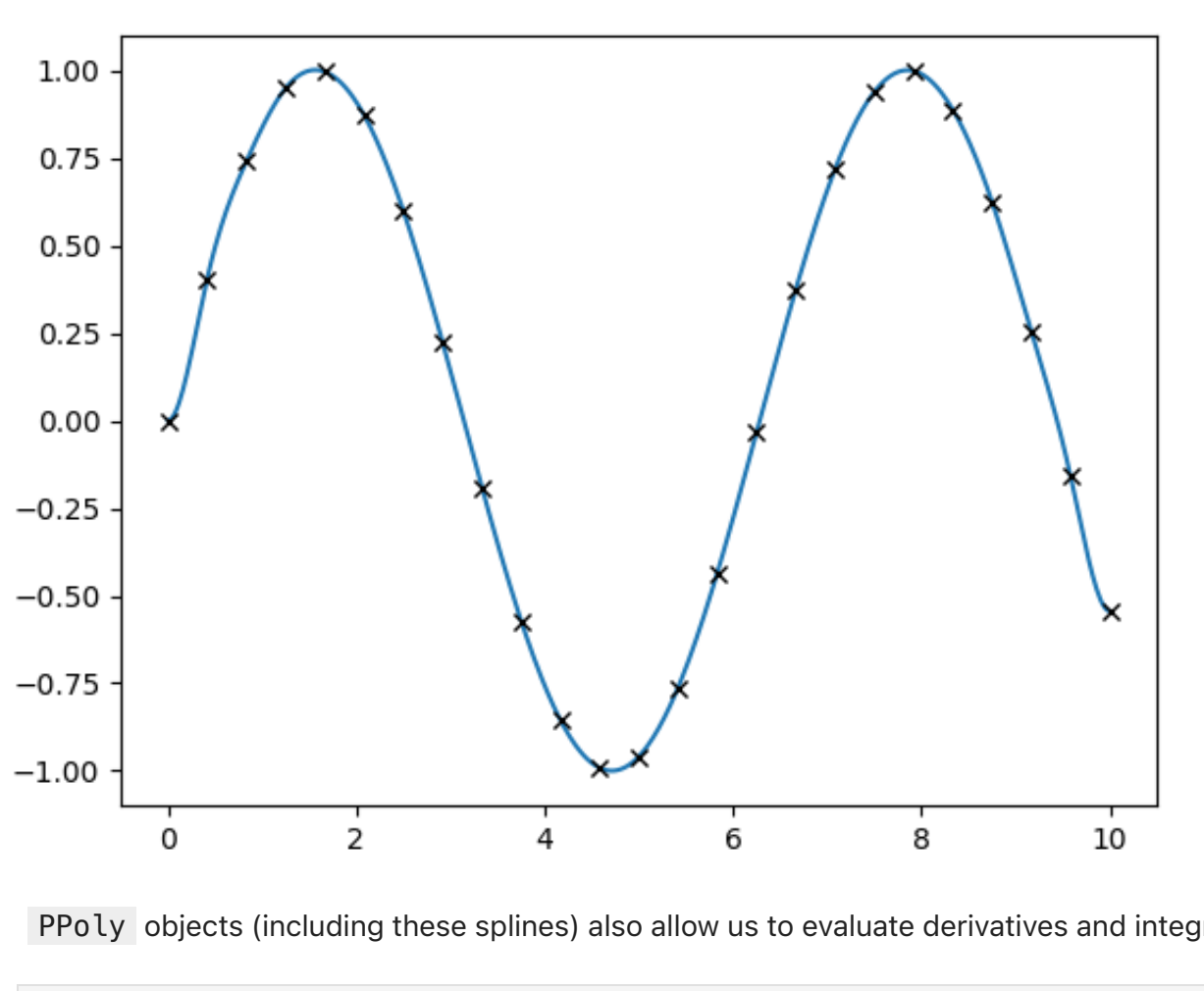
Out[6]:
```
[<matplotlib.lines.Line2D at 0x1235956d0>]
```



By default, using "not-a-knot" spline boundary condition, but can also specify periodic or clamped

In [7]:
```
gspline = interp.CubicSpline(x,y, bc_type = 'clamped')
y_fine = gspline(x_fine)
plt.plot(x_fine, y_fine)
plt.plot(x, y, 'kx ')
```

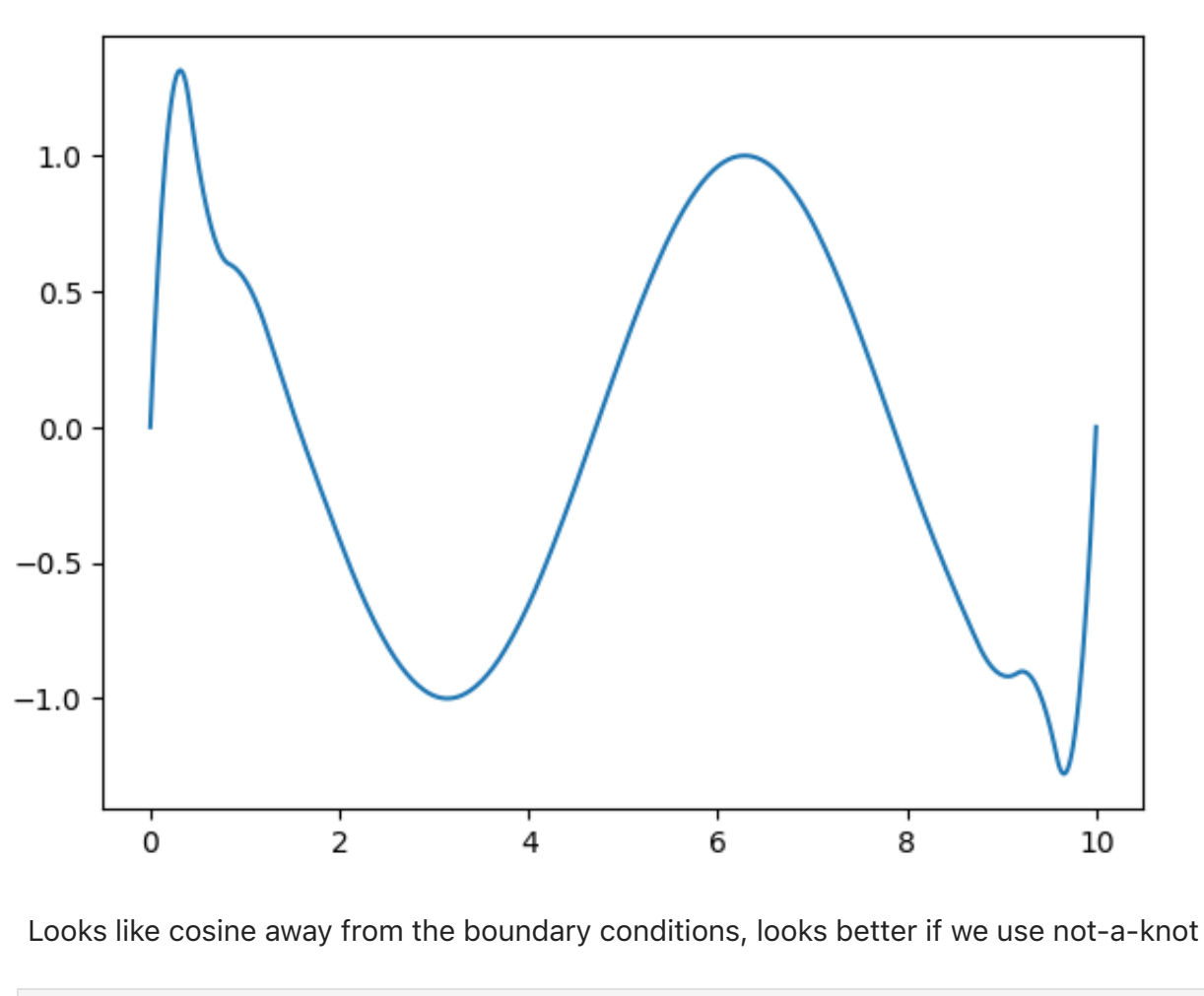
Out[7]:
```
[<matplotlib.lines.Line2D at 0x12361ccd0>]
```



`PPoly` objects (including these splines) also allow us to evaluate derivatives and integrals of our interpolants

In [8]:
```
dg = gspline.derivative()
d_fine = dg(x_fine)
plt.plot(x_fine, d_fine)
```

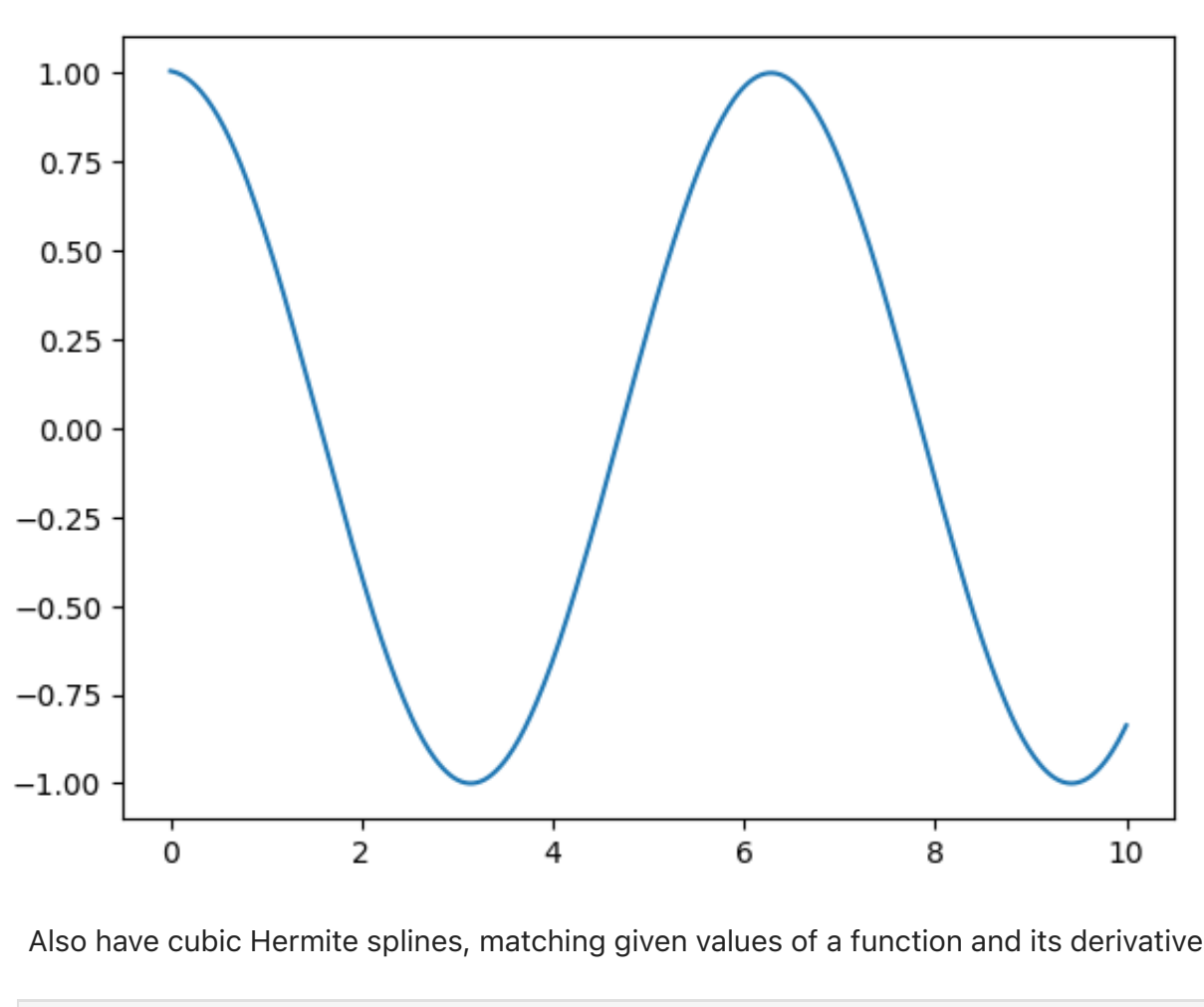
Out[8]:
```
[<matplotlib.lines.Line2D at 0x1236b4190>]
```



Looks like cosine away from the boundary conditions, looks better if we use not-a-knot `bc_type`

In [9]:
```
gspline = interp.CubicSpline(x,y)
dg = gspline.derivative()
d_fine = dg(x_fine)
plt.plot(x_fine, d_fine)
```

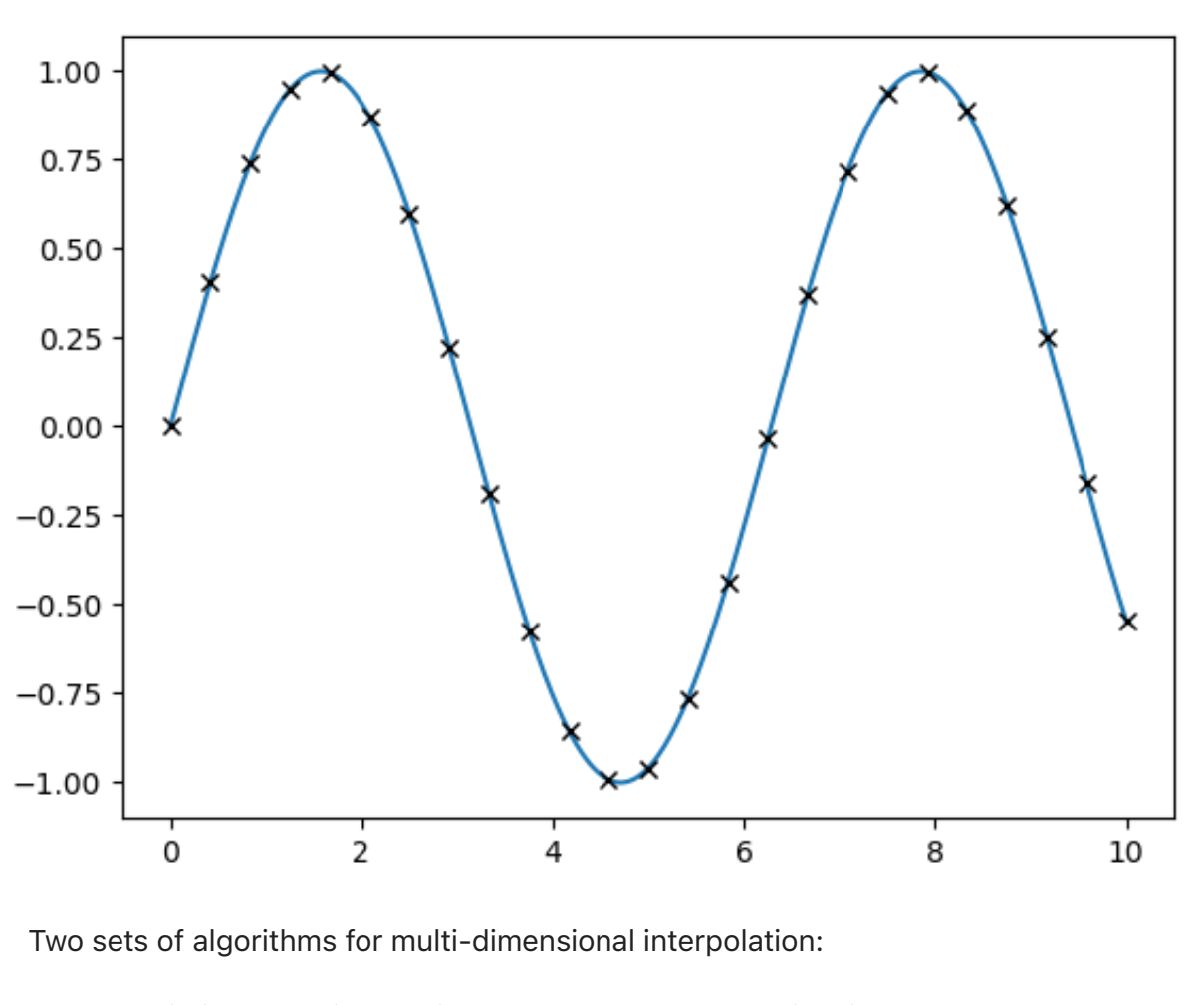
Out[9]:
```
[<matplotlib.lines.Line2D at 0x123709810>]
```



Also have cubic Hermite splines, matching given values of a function and its derivatives at a point

In [10]:
```
dy = np.cos(x)
gHermite = interp.CubicHermiteSpline(x, y, dy)
y_fine = gHermite(x_fine)
plt.plot(x_fine, y_fine)
plt.plot(x, y, 'kx ')
```

Out[10]:
```
[<matplotlib.lines.Line2D at 0x123790e10>]
```



Two sets of algorithms for multi-dimensional interpolation:

Regular grid interpolation routines that take arrays of points in x, y, z, etc., and data on tensor-product meshes of these points

In [11]:
```
x = np.array([0.0, 0.3, 0.5, 0.6, 1.0])
y = np.linspace(2,3,7)

X, Y = np.meshgrid(x, y, indexing='ij')

print(X)
print(Y)
```

```
[[0.  0.  0.  0.  0.  0.  0. ]
 [0.3 0.3 0.3 0.3 0.3 0.3 0.3]
 [0.5 0.5 0.5 0.5 0.5 0.5 0.5]
 [0.6 0.6 0.6 0.6 0.6 0.6 0.6]
 [1.  1.  1.  1.  1.  1.  1. ]]
[[2.         2.16666667 2.33333333 2.5        2.66666667 2.83333333
  3.        ]
 [2.         2.16666667 2.33333333 2.5        2.66666667 2.83333333
  3.        ]
 [2.         2.16666667 2.33333333 2.5        2.66666667 2.83333333
  3.        ]
 [2.         2.16666667 2.33333333 2.5        2.66666667 2.83333333
  3.        ]
 [2.         2.16666667 2.33333333 2.5        2.66666667 2.83333333
  3.        ]]
```

`meshgrid` takes two 1D arrays and produces 2D arrays that match given data points

- 2 options for `meshgrid` output, need this one (`indexing = "ij"`) to match what `RegularGridInterpolator` expects for its input

In [14]:
```
F = np.sin(np.pi*X)*np.exp(Y)
F_interp = interp.RegularGridInterpolator((x,y), F)

print(f"At (0.1, 2.1), f(x,y) = {np.sin(np.pi*0.1)*np.exp(2.1)}, interpolated value is {F_interp((0.1, 2.1))}")
```

```
At (0.1, 2.1), f(x,y) = 2.523485281936783, interpolated value is 2.2094538506899486
```

In [15]:
```
print(f"At (0.51, 2.15), f(x,y) = {np.sin(np.pi*0.51)*np.exp(2.15)}, interpolated value is {F_interp((0.51, 2.15))}")
```

```
At (0.51, 2.15), f(x,y) = 8.580622287790144, interpolated value is 8.553062576059872
```

If we have function that we can sample on the regular grid (e.g., as generated by `np.meshgrid`), then `RegularGridInterpolator` gives us good functionality.

More common to have unstructured input data, where we simply have a list of coordinates and data values that we want to interpolate.

In 2D, have `LinearNDInterpolator` that constructs a triangulation for given set of (x, y) points, then does piecewise linear interpolation on each triangle. Also have `RBFInterpolator` that uses radial basis functions to do interpolation of higher order on unstructured data.

Some of these extend to even more dimensions...

`scipy.integrate` provides functions for numerical integration in 2 contexts, for evaluating definite integrals, and for approximating solutions of systems of ODEs

In 1D, standard routine for definite integrals is `quad`:

In [16]:
```
import scipy.integrate as integrate
f = lambda x: x**2
integrate.quad(f, 0, 1)
```

Out[16]:
```
(0.3333333333333333, 3.700743415417188e-15)
```

Notes:

- using a lambda function here, just an easy way to create a simple function in python
- `integrate.quad` returns 2 outputs, one being the approximated value of the integral and the second being an error estimate.
- `integrate.quad` uses quite a complex adaptive algorithm to achieve high orders of accuracy. This is good when we have complex functions to integrate, but can be expensive when we're integrating simple functions

If we have data evaluated on a grid of points, `integrate.trapezoid` and `integrate.simpson` implement standard trapezoidal rule and Simpson's rule:

In [17]:
```
x = np.linspace(0,1,25)
y = x**2
integrate.trapezoid(y, x)
```

Out[17]:
```
np.float64(0.3336226851851851)
```

In [18]:
```
integrate.simpson(y, x)
```

Out[18]:
```
np.float64(0.33333333333333337)
```

Be careful: take y values first, then x values

Alternatively, can use simpler Gaussian quadrature routines:

In [19]:
```
integrate.fixed_quad(f, 0, 1, n=3)
```

Out[19]:
```
(np.float64(0.33333333333333337), None)
```

This uses fixed order quadrature, integrating polynomial functions up to order n (or $n+1$ if n even) exactly:

In [21]:
```
for n in [3, 5, 7, 9]:
    val = integrate.fixed_quad(lambda x: 12*x**11 - 4*x**3, 1, 2, n=n)
    print(f"With order {n}, approximation is {val[0]}")
```

```
With order 3, approximation is 4063.934212500002
With order 5, approximation is 4079.99971655329
With order 7, approximation is 4079.999999999999
With order 9, approximation is 4080.000000000008
```

For multiple integrals, `quad` has extensions `dblquad`, `tplquad`, and `nquad`

Note: `dblquad` takes function $f(y, x)$ and integrates from $x = a$ to b, and $y = g_1(x)$ to $g_2(x)$ with

```
integrate.dblquad(f, a, b, g1, g2)
```

but `nquad` takes integration bounds in the opposite order and with different syntax

Consider $\int_0^1 \int_x^{2-x} (x+1) y dy dx = \frac{4}{3}$

In [22]:
```
f = lambda y,x: (x+1)*y
integrate.dblquad(f, 0, 1, lambda x:x, lambda x: 2-x)
```

Out[22]:
```
(1.3333333333333333, 2.2204355797178593e-14)
```

In [23]:
```
def g(x):
    return [x, 2-x]
integrate.nquad(f, [g, [0,1]])
```

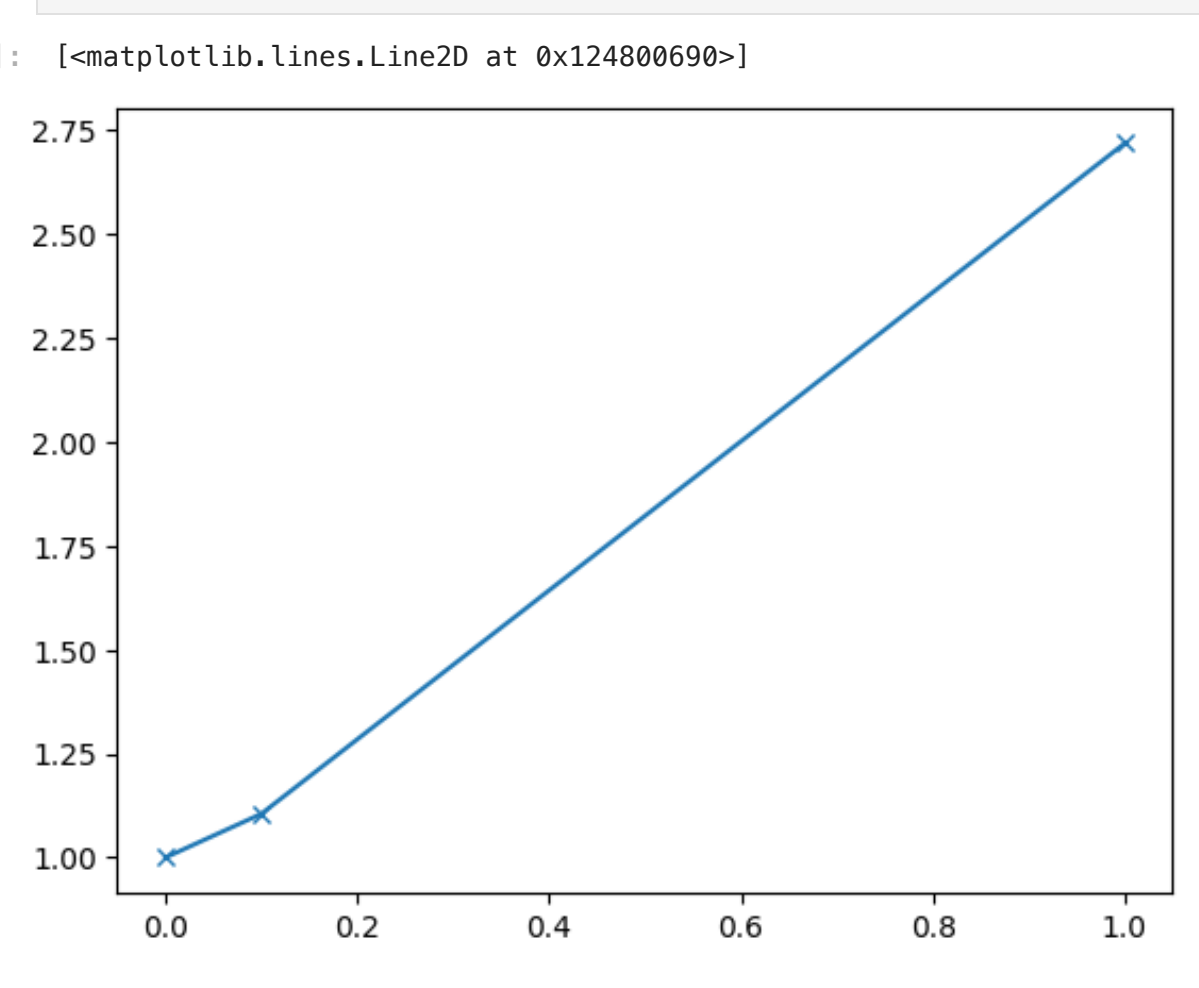
Out[23]:
```
(1.3333333333333333, 2.2204355797178593e-14)
```

`integrate.solve_ivp` gives numerical solution to $\frac{dy}{dt} = f(t, y)$ with $y(t_0) = y_0$

Basic functionality is designed to ensure good accuracy at a fixed time point, t_1

In [24]:
```
def f(t,y):
    return y
out = integrate.solve_ivp(f, [0,1], [1])
plt.plot(out.t, out.y[0], 'x-')
```

Out[24]:
```
[<matplotlib.lines.Line2D at 0x124800690>]
```



Output data struture from `solve_ivp` has a 1D array of times, `t`, at which approximations are computed, and a 2D array of values of `y` at these times:

In [25]:
```
print(out)
```

```
 message: The solver successfully reached the end of the integration interval.
 success: True
  status: 0
       t: [ 0.000e+00  1.000e-01  1.000e+00]
       y: [[ 1.000e+00  1.105e+00  2.718e+00]]
     sol: None
t_events: None
y_events: None
    nfev: 14
    njev: 0
     nlu: 0
```

This allows us to solve with multiple initial conditions at once:

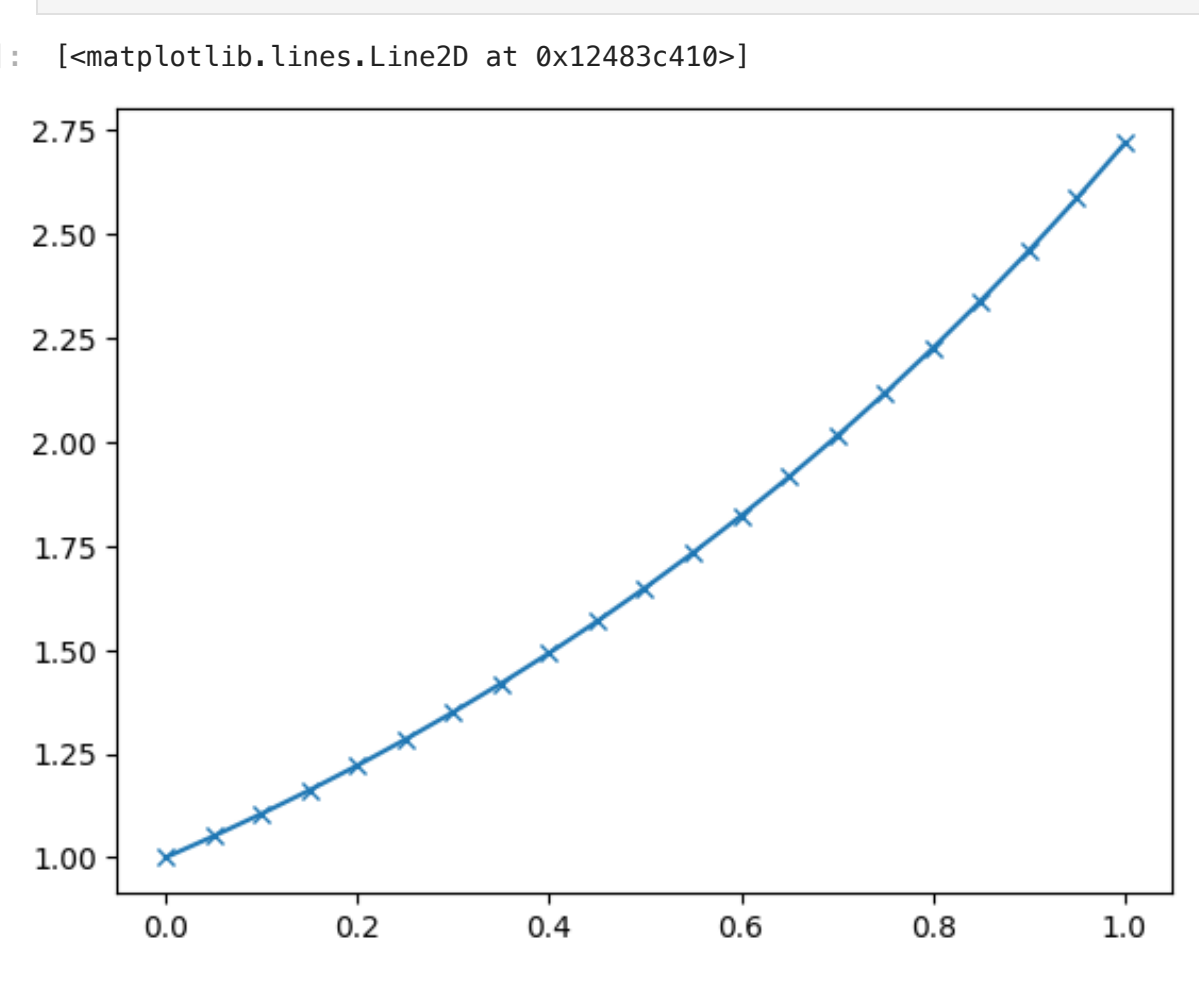
In [27]:
```
def f(t,y):
    return y
out = integrate.solve_ivp(f, [0,1], [1, 2])
print(out)
```

```
 message: The solver successfully reached the end of the integration interval.
 success: True
  status: 0
       t: [ 0.000e+00  1.000e-01  1.000e+00]
       y: [[ 1.000e+00  1.105e+00  2.718e+00]
           [ 2.000e+00  2.210e+00  5.437e+00]]
     sol: None
t_events: None
y_events: None
    nfev: 14
    njev: 0
     nlu: 0
```

Adaptive method here is good if we just want that final time accuracy, but if we want better accuracy over the interval, can set `max_step` to limit step size and run again:

In [28]:
```
out = integrate.solve_ivp(f, [0,1], [1], max_step = 0.05)
plt.plot(out.t, out.y[0], 'x-')
```

Out[28]:
```
[<matplotlib.lines.Line2D at 0x12483c410>]
```



Many choices of methods here:

- `RK45` is default method, uses 2 explicit Runge-Kutta schemes, of orders 4 and 5, to control error
- `Radau` uses an implicit 5th order RK method with similar error estimation
- Also some lower/higher order schemes available

`scipy.optimize` provides routines for constrained and unconstrained minimization problems.

Several key classes of problems. Start with unconsrained minimization: given a function $f : \mathbb{R}^n \to \mathbb{R}$, find $\vec{x}$ such that $f(\vec{x})$ is minimized

A standard test problem for optimization routines is known as the "Rosenbrock function":

$$f(\vec{x}) = \sum_{i=1}^{n-1} 100(x_{i+1} - x_i)^2 + (1 - x_i)^2$$

for vectors, $\vec{x}$, of length n

The minimum of this function is 0 when $x_i = 0$ for $1 \le i \le n$

Standard routine for minimization is `minimize` which takes a function to minimize and an initial guess for the minimizer:

In [30]:
```
import scipy.optimize as optim
from rosenbrock import *

x0= np.zeros(5)
out = optim.minimize(rosenbrock, x0)
print(out)
```

```
 message: Optimization terminated successfully.
 success: True
  status: 0
     fun: 5.370412703770336e-11
       x: [ 1.000e+00  1.000e+00  1.000e+00  1.000e+00  1.000e+00]
     nit: 39
     jac: [ 3.001e-06 -4.512e-06  5.418e-06 -5.464e-06  1.709e-06]
hess_inv: [[ 7.580e-03  1.260e-02 ...  4.744e-02  9.476e-02]
           [ 1.260e-02  2.518e-02 ...  9.478e-02  1.892e-01]
           ...
           [ 4.744e-02  9.478e-02 ...  3.805e-01  7.597e-01]
           [ 9.476e-02  1.892e-01 ...  7.597e-01  1.522e+00]]
    nfev: 294
    njev: 49
```

Lots of information here. Note:

- `out.x` is the solution. Looks pretty good...

In [31]:
```
out.x - np.ones(5)
```

Out[31]:
```
array([-8.01802492e-07, -1.60017682e-06, -3.17669430e-06, -6.35231579e-06,
       -1.27034988e-05])
```

- Took 39 iterations (`nit`), 294 function evaluations `nfev`, 49 Jacobian evaluations `njev`

Default minimization algorithm is the "BFGS"

To minimize a general function, $f(\vec{x})$, we generally think of applying Newton's method to "the first-order optimality conditions". That $\nabla f(\vec{x}) = \vec{0}$ when $\vec{x}$ is a minimizer

In this context, Newton's method would take an initial guess, $\vec{x}_0$, and solve

$$H_k(\vec{x}_{k+1} - \vec{x}_k) = -\nabla f(\vec{x}_k)$$

for $k = 0, 1, \ldots$, where H_k is the Hessian of f (the matrix of its second derivatives), to generate a sequence of approximations, $\vec{x}_0$, $\vec{x}_1$, $\vec{x}_2$, ... to the minimizer

Applying Newton's method then requires computing or approximating both $\nabla f(\vec{x}_k)$ and H_k at each iteration.

The standard way to estimate $\nabla f(\vec{x}_k)$ is through a finite-difference approximation, where we approximate each component of $\nabla f(\vec{x}_k)$ by

$$\frac{\partial f}{\partial x_j} \approx \frac{1}{h}(f(\vec{x}_k + h\vec{e}_j) - f(\vec{x}_k))$$

for $1 \le j \le n$, where $\vec{e}_j$ is the j^{th} canonical unit vector (with entry 1 in position j and zero otherwise)

BFGS uses such approximations to also approximate the Hessian in a reasonable way

Note that in the above example, we need 6 function evaluates to approximate the 5 components of $\nabla f(\vec{x}_k)$, and we used 49 Jacobian evaluations and 294 function evaluations, noting $49 \times 6 = 294$

This means that all of our function evaluations were used to estimating the gradient

Next week, look at other methods for minimization that use more/different information